

2η Προγραμματιστική Εργασία - Αναφορά

Βασίλης Νικολάου – 3230141

Γιώργος Παπαχρήστος – 3230156

Ευθύμης Ποπώλης - 3230168

Μέρος Α

Σκοπός του πρώτου μέρους της εργασίας είναι η χρήση classifiers για την ταξινόμηση κριτικών ταινιών από το IMDB Dataset σε θετικές ή αρνητικές.

Λεξιλόγιο, Υπερ-Παράμετροι και Δεδομένα

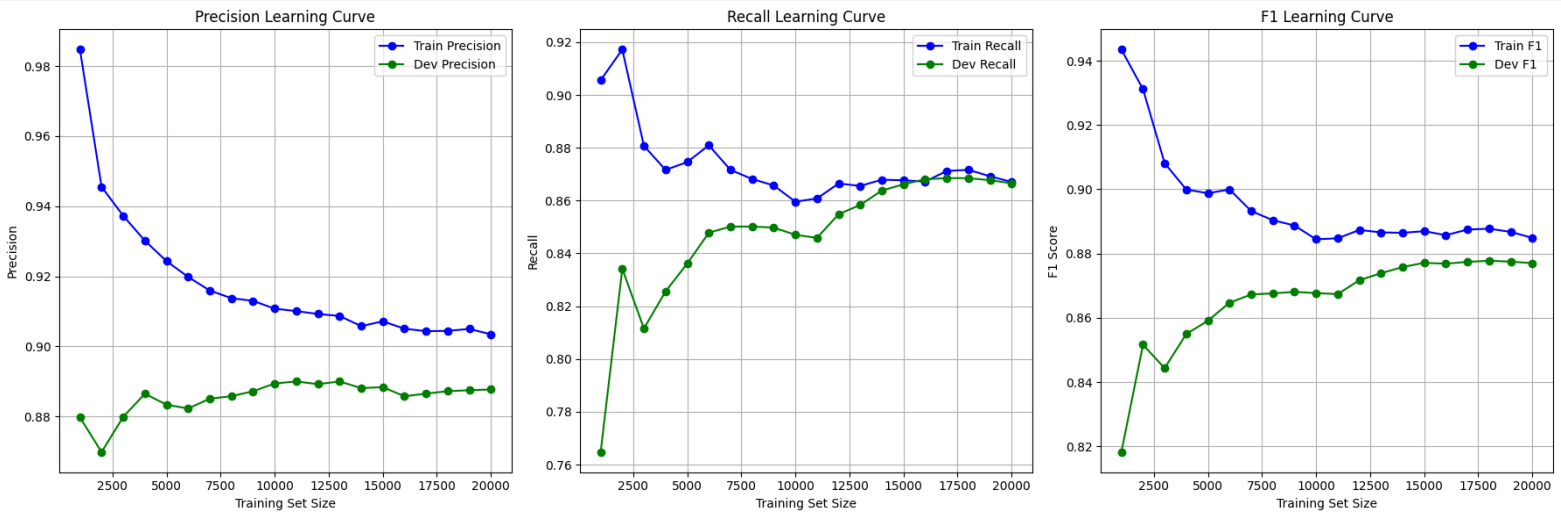
Αφού φορτώσαμε το dataset, υπολογίσαμε τη συχνότητα εμφάνισης των λέξεων και τις ταξινομήσαμε βάσει αυτής. Από αυτές τις λέξεις, αγνοήσαμε τις $n = 20$ πιο συχνές και τις $k = 50.000$ πιο σπάνιες. Τα n και k έγιναν fine-tuned: το n είναι ίσο με 20 γιατί αν ήταν μεγαλύτερο, η λέξη "not" θα ήταν εκτός λεξιλογίου και είναι σημαντική για την κατηγοριοποίηση των κριτικών. Στη συνέχεια, αναπαραστήσαμε κάθε κριτική σαν ένα διάνυσμα όπου σε κάθε θέση έχει τιμή 1 ή 0, ανάλογα αν η αντίστοιχη λέξη εμφανίζεται ή όχι στην κριτική. Έπειτα, με τις περίπου 30.000 λέξεις που απέμειναν υπολογίσαμε το Information Gain κάθε λέξης και τις ταξινομήσαμε βάσει αυτού. Από αυτές, κρατήσαμε τις $m = 8.000$ λέξεις με το μεγαλύτερο Information Gain και έτσι καταλήξαμε με διανύσματα 8.000×1 για κάθε κριτική. Τέλος, χωρίσαμε τα δεδομένα σε 80% training και 20% development για την εκπαίδευση των μοντέλων μας και ορίσαμε τις υπερ-παραμέτρους.

Ο **Naive Bayes** χρησιμοποιεί πολυωνυμική μορφή Bernoulli με εκτιμήτρια Laplace για να μην υπάρχει ο κίνδυνος μηδενικών πιθανοτήτων ($\alpha = 0.1$ προκαθορισμένη).

RandomForestClassifier(criterion='entropy', n_estimators=500, random_state = 42): Χρησιμοποιώντας την παράμετρο criterion='entropy', καθώς δουλεύουμε με δυαδικούς αριθμούς, ο RandomForestClassifier θα λειτουργεί ακριβώς όπως ο αλγόριθμος ID3. Επιπλέον, κάναμε fine-tune τον αριθμό των δέντρων που θα παραχθούν στο δάσος (n_estimators=500) και χρησιμοποιήσαμε random_state = 42 για να εξασφαλίσουμε ότι τα αποτελέσματα θα μπορούν να αναπαραχθούν χωρίς αποκλίσεις.

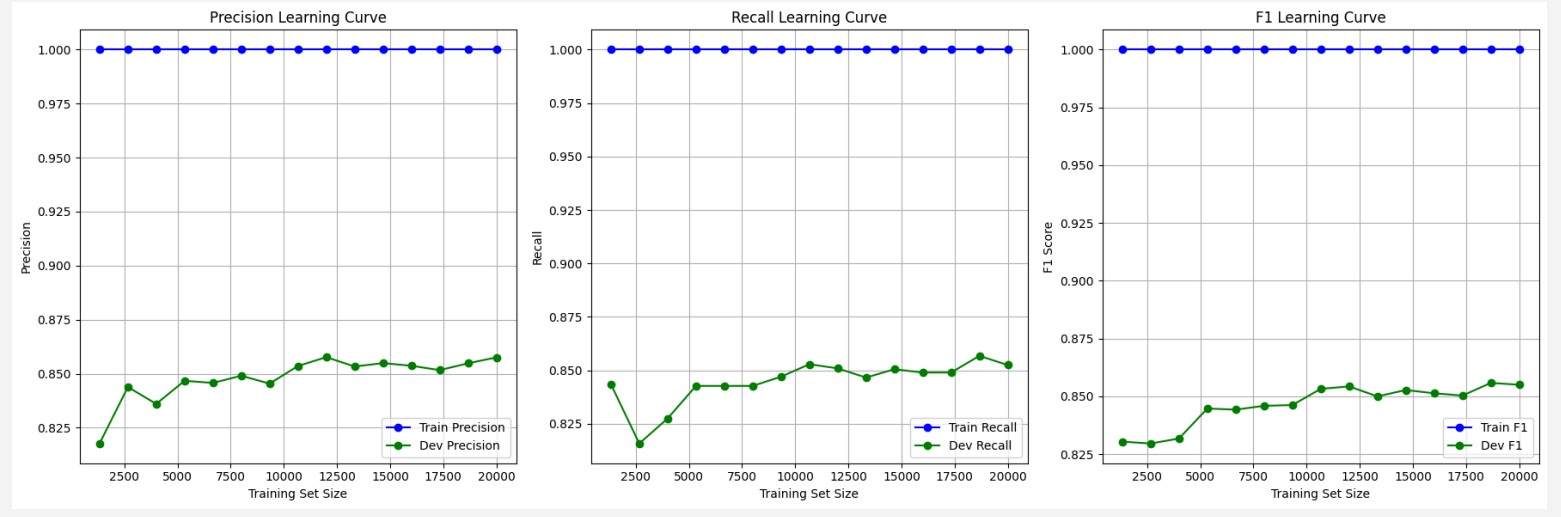
Ο **AdaBoostClassifier** χρησιμοποιεί 250 estimators πετυχαίνοντας ικανοποιητική ακρίβεια σε λογικό χρονικό διάστημα. Η αύξηση των estimators θα έδινε μεγαλύτερη ακρίβεια αλλά ο κώδικας θα έτρεχε για πολλή περισσότερη ώρα λόγω της εκθετικής αύξησης του χρόνου.

1) (Multinomial) Naive Bayes Classifier



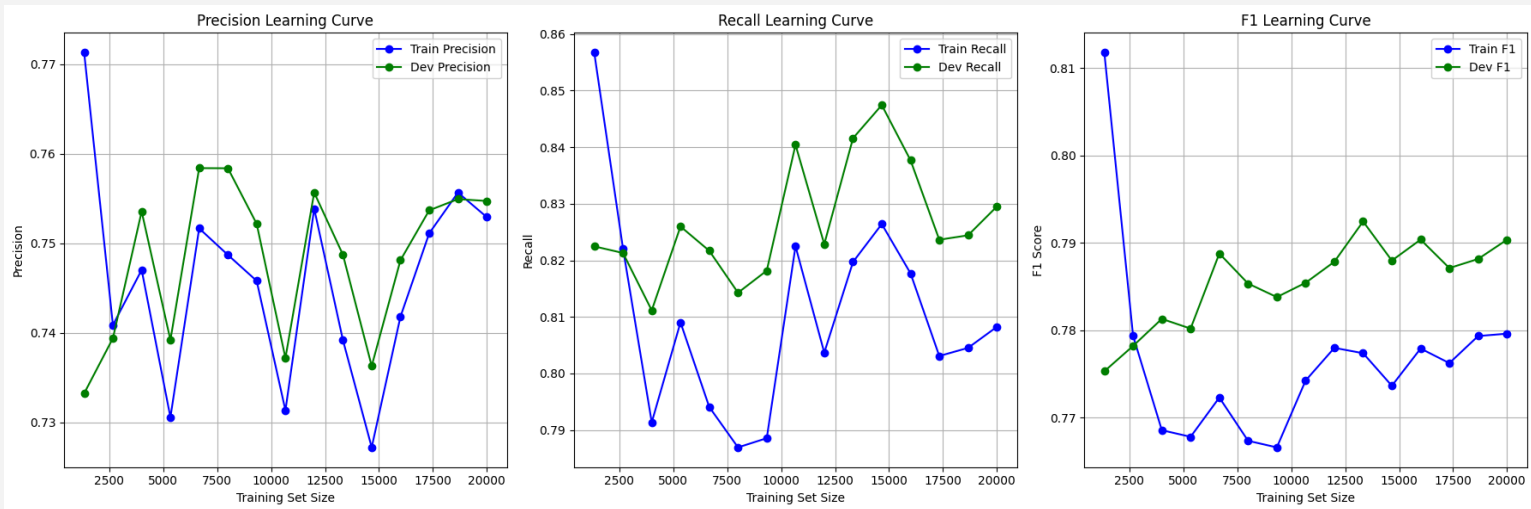
...	precision	recall	f1-score	support
0	0.81	0.88	0.85	12500
1	0.87	0.80	0.83	12500
accuracy			0.84	25000
macro avg	0.84	0.84	0.84	25000
weighted avg	0.84	0.84	0.84	25000

2) Random Forest Classifier



...	precision	recall	f1-score	support
0	0.86	0.86	0.86	12500
1	0.86	0.86	0.86	12500
accuracy			0.86	25000
macro avg	0.86	0.86	0.86	25000
weighted avg	0.86	0.86	0.86	25000

3) AdaBoost Classifier



	precision	recall	f1-score	support
0	0.84	0.79	0.81	12500
1	0.80	0.84	0.82	12500
accuracy			0.82	25000
macro avg	0.82	0.82	0.82	25000
weighted avg	0.82	0.82	0.82	25000

Μέρος Β

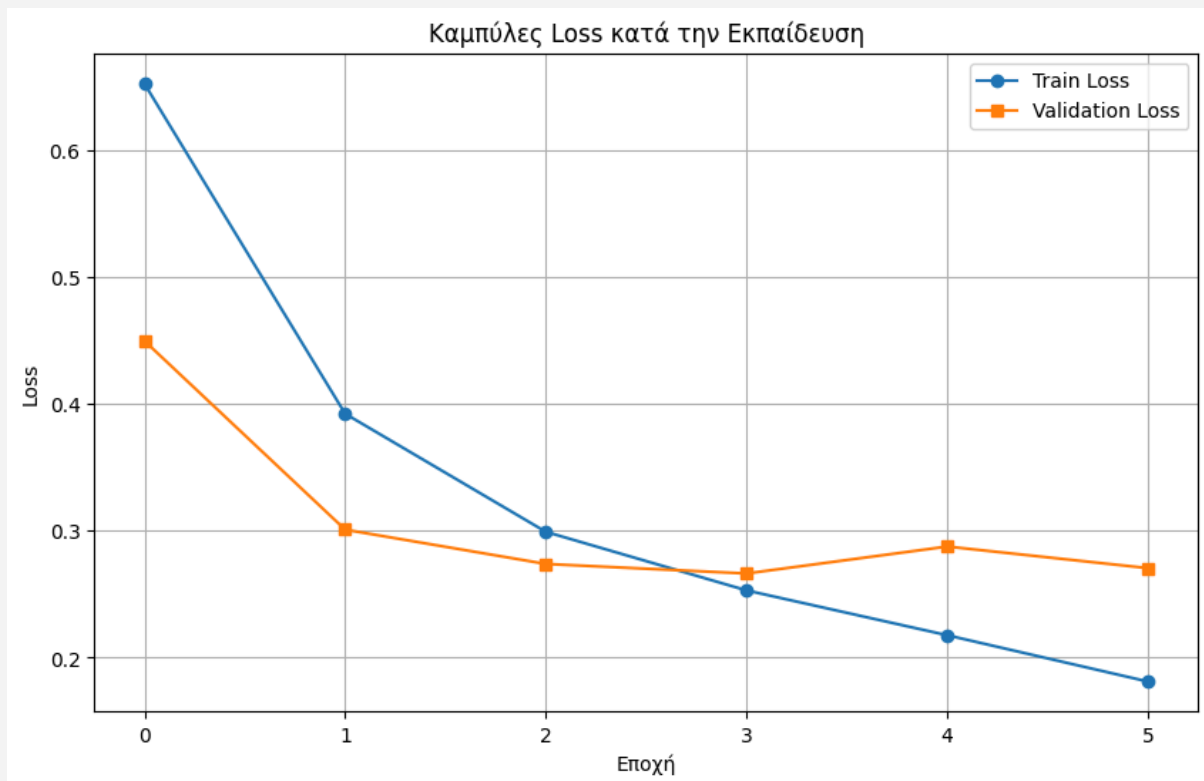
Σκοπός του δεύτερου μέρους της εργασίας είναι η σύγκριση των αποτελεσμάτων των ταξινομητών του μέρους Α με ένα σπιβαγμένο RNN διπλής κατεύθυνσης με LSTM κελιά.

Μοντέλο

Πρώτα εγκαθιστούμε το Pytorch και κάνουμε τα κατάλληλα imports. Συνεχίζουμε φορτώνοντας τις κριτικές ταινιών, τα word embeddings από το GloVe και δημιουργούμε το λεξιλόγιο μαζί με τα word embeddings για τις λέξεις που περιέχονται σε αυτό. Χρησιμοποιήθηκαν τρία επίπεδα bidirectional LSTM για εκμάθηση πιο σύνθετων χαρακτηριστικών του κειμένου. Αξιοποιήσαμε self-attention MLP για τον υπολογισμό των βαρών στις κρυφές καταστάσεις του LSTM, ώστε η τελική απόφαση να κρίνεται από τις πιο σημαντικές λέξεις. Η τελική ταξινόμηση (0 ή 1) προκύπτει από το αποτέλεσμα του attention με εφαρμογή sigmoid συνάρτησης ενεργοποίησης. Εφαρμόσαμε και dropout για την αποφυγή overfitting.

Δεδομένα και Υπερ-παράμετροι

Κρατήσαμε τις 15.000 πιο συχνές λέξεις και εφαρμόστηκε padding στις κριτικές για να έχουν σταθερό μήκος 400 λέξεις. Χωρίσαμε τα δεδομένα σε 80% train, 10% development, 10% test. Τα word embeddings είναι διανύσματα διαστάσεων 200x1 και το πλήθος των κρυφών καταστάσεων 192. Το dropout rate ήταν 0.5, το batch size 64 και το learning rate στον Adam optimizer 0.0005. Τέλος, το μέγιστο πλήθος εποχών ήταν 15 και εφαρμόσαμε και early stopping για την εκπαίδευση. Η εκπαίδευση έγινε βάσει του validation loss για να μην έχουμε overfitting. Το μοντέλο σταμάτησε την εκπαίδευση στην 8η εποχή επειδή το validation loss εμφάνισε αύξηση μετά την 6η εποχή (ενεργοποιώντας το early stopping).



Παρατηρούμε ότι το **RNN πέτυχε καλύτερο F1 score** από τους αλγορίθμους του Μέρους Α. Αυτό συμβαίνει λόγω της ικανότητας των LSTM να εξαγάγουν το νόημα του κειμένου μέσω των ενθέσεων λέξεων.

Αποτελέσματα			
Loss: 0.2534			
Accuracy: 0.8924			
Αποτελέσματα Για Κάθε Κλάση			
Κλάση	Precision	Recall	F1-Score
positive	0.8813	0.9111	0.8959
negative	0.9047	0.8731	0.8886
Κύριες Μετρικές			
Μέτρηση	Τιμή		
Macro Precision	0.8930		
Macro Recall	0.8921		
Macro F1	0.8923		
Micro Precision	0.8924		
Micro Recall	0.8924		
Micro F1	0.8924		

Μέρος Γ

Σκοπός του τρίτου μέρους της εργασίας ήταν η ταξινόμηση εικόνων από το σύνολο δεδομένων FashionMNIST χρησιμοποιώντας έναν κωδικοποιητή CNN συνδεδεμένο με MLP, ο οποίος θα υλοποιηθεί σε Pytorch.

Imports και Προεπεξεργασία

Αρχικά, κάνουμε import τις βιβλιοθήκες της Pytorch, τις βιβλιοθήκες για τις μετρικές και τις βιβλιοθήκες για τα γραφήματα. Συνεχίζουμε, θέτοντας το σύστημα σε GPU, αν γίνεται, για να τρέξει πιο γρήγορα το μοντέλο και ορίζουμε μία συνάρτηση προεπεξεργασίας στα pixel των εικόνων της βάσης λόγω του προεκπαιδευμένου μοντέλου που θα χρησιμοποιήσουμε στην συνέχεια.

Data Loaders

Ορίζουμε την συνάρτηση για τους data loaders η οποία δουλεύει ως εξής: ελέγχει ότι η κανονικοποίηση έγινε σωστά (έχουμε, δηλαδή, 3 κανάλια, το μέγεθος των εικόνων) και κάνει επαύξηση στα δεδομένα στρίβοντας και φλιπάροντας τις εικόνες. Αυτό που επιστρέφει είναι τα δεδομένα εκπαίδευσης, επικύρωσης και αξιολόγησης, χρησιμοποιώντας την έτοιμη κλάση Dataloader που προσφέρει η Pytorch.

Resnet Μοντέλο

Για την συνάρτηση του μοντέλου ελέγχουμε αν είναι pretrained (όταν το καλούμε είναι), έτσι ώστε να χρησιμοποιήσουμε τα βάρη που έχει ήδη, ορίζουμε 256 νευρώνες στα κρυφά επίπεδα, dropout rate 0,5 και να χρησιμοποιεί συνδεδεμένο MLP.

Εκπαίδευση και Αξιολόγηση

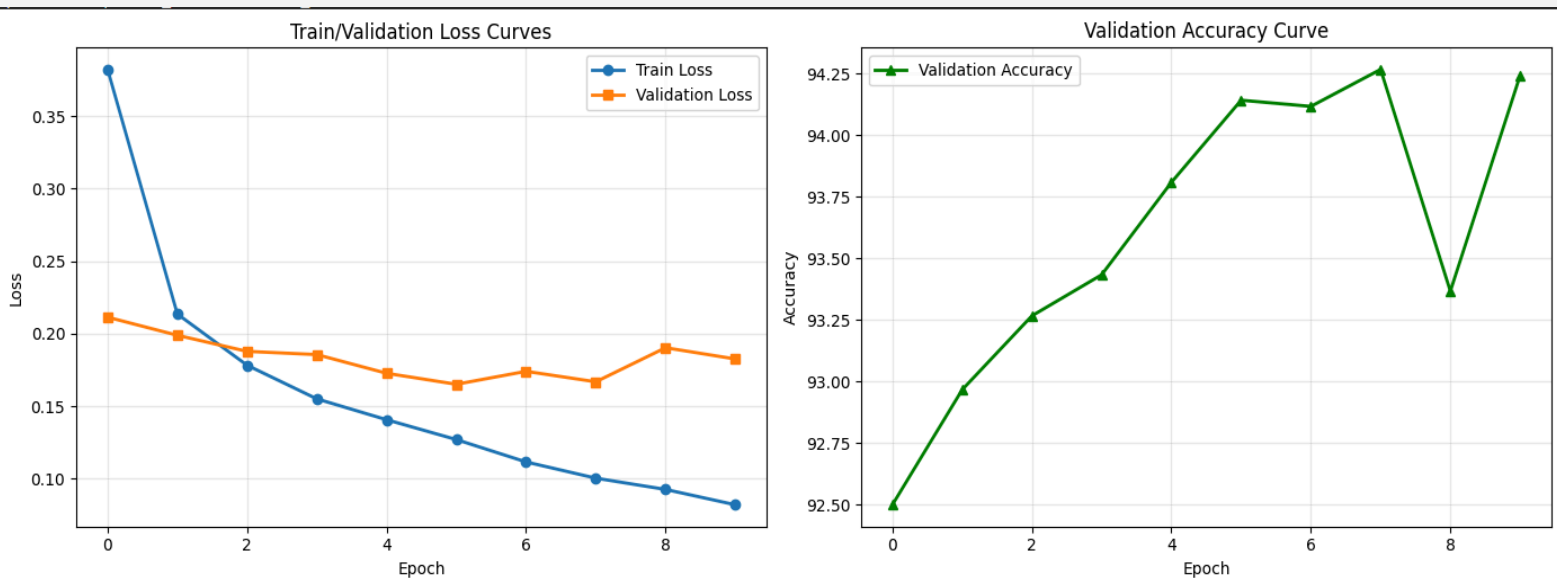
Ορίζουμε την κλάση RunConfig για να περιέχει τις μεταβλητές για την εκπαίδευση όπως την εποχή, το weight decay κλπ. Στην συνέχεια ορίζουμε την συνάρτηση εκπαίδευσης, η οποία για κάθε εποχή εκπαιδεύει το μοντέλο σε έναν συγκεκριμένο αριθμό από batches που ορίζει η RunConfig, κρατώντας το loss για τα δεδομένα εκπαίδευσης και τα δεδομένα επικύρωσης. Επίσης, κρατάει το καλύτερο μοντέλο από όλες τις εποχές με βάση το accuracy στα δεδομένα επικύρωσης και κάνει early stopping με υπομονή 3 εποχές ως default τιμή. Τέλος, ορίζουμε την συνάρτηση αξιολόγησης η οποία υπολογίζει (χρησιμοποιώντας την συνάρτηση precision_recall_fscore_support της βιβλιοθήκης sklearn) και επιστρέφει το recall, το precision και το f1 score για κάθε κλάση και τους micro και macro μέσους όρους αυτών.

Καμπύλες και Πίνακες Μετρήσεων

Για την καμπύλη του Loss στα δεδομένα εκπαίδευσης και επικύρωσης, ορίσαμε την συνάρτηση `plot_loss_curves` η οποία δημιουργεί το γράφημα με τα losses αλλά και επιπλέον ένα γράφημα για το accuracy του μοντέλου σε κάθε εποχή, γιατί παρότι δεν το αναφέρει η εκφώνηση το θεωρήσαμε σημαντικό να συμπεριληφθεί αφού με βάση αυτό διαλέξαμε το καλύτερο μοντέλο. Για τους πίνακες με τις μετρικές ορίσαμε την `print_metrics_tables`, η οποία εμφανίζει τα δεδομένα που επιστρέφει η συνάρτηση αξιολόγησης με συγκεκριμένη μορφοποίηση.

Main

Η κύρια συνάρτηση καλεί όλες τις παραπάνω συναρτήσεις που ορίσαμε, δηλαδή καλεί την συνάρτηση για τους data loaders και τα δεδομένα αυτά τα βάζει ως είσοδο στην εκπαίδευση και την αξιολόγηση του μοντέλου, που αρχικοποιεί αμέσως μετά το φόρτωμα των δεδομένων. Στην συνέχεια, εμφανίζει τα αποτελέσματα που προέκυψαν από την εκπαίδευση και την αξιολόγηση, καλώντας τις αντίστοιχες συναρτήσεις.



Στον παρακάτω πίνακα φαίνονται οι μέσοι όροι στις μετρικές και οι τιμές του recall, του precision και του f1-score ανά κλάση. Από τις τιμές αυτές παρατηρούμε πως το μοντέλο μας έχει πολύ καλή επίδοση στις περισσότερες κλάσεις εκτός από τα t-shirts και τα shirts.

```
Βασικές μετρικές test set:  
Accuracy: 0.9430  
Precision (macro): 0.9429  
Recall (macro): 0.9430  
F1 (macro): 0.9428
```

```
=====
```

Αποτελέσματα Για Κάθε Κλάση

```
=====
```

Κλάση	Precision	Recall	F1-Score
T-shirt/top	0.8925	0.8970	0.8948
Trouser	0.9970	0.9880	0.9925
Pullover	0.9285	0.9090	0.9186
Dress	0.9199	0.9650	0.9419
Coat	0.9263	0.9180	0.9221
Sandal	0.9804	0.9980	0.9891
Shirt	0.8374	0.8190	0.8281
Sneaker	0.9797	0.9640	0.9718
Bag	0.9910	0.9960	0.9935
Ankle boot	0.9760	0.9760	0.9760

```
-----
```

```
=====
```

Κύριες Μετρικές

```
=====
```

Μέτρηση	Τιμή
Accuracy	0.9430
Macro Precision	0.9429
Macro Recall	0.9430
Macro F1	0.9428
Micro Precision	0.9430
Micro Recall	0.9430
Micro F1	0.9430

```
-----
```

Οι υπερ-παράμετροι που χρησιμοποιήθηκαν είναι οι παρακάτω:

Μέγεθος εικόνας: 224x224
Batch size: 64
Weight decay: 0.0001
Learning rate: 0.0001
Εποχές: 10
Optimizer: Adam
Criterion: CrossEntropyLoss